

RAG Assistant PoC

Solution design for a retrieval-augmented question answering service over internal documents. Problem and scope first, then five views of the same system, the user-facing behaviour, the API contract, and the decisions, requirements, operations and risks behind them.

Owner Jesus Roncal **Updated** September 2026 **Status** Draft for review **Version** 0.7 **Notation** C4 model, UML sequence
Scope Proof of concept, single tenant

 System in scope

 Container or infrastructure node

 External system or person

 Control point

 Measurement and governance

1 Problem, goals and scope

Employees and support agents spend time searching policies, FAQs and manuals, and answers given from memory are inconsistent. The proof of concept tests whether a retrieval-augmented assistant can answer these questions with citations, decline safely when the documents do not support an answer, and do so at a predictable cost.

Goals	Out of scope for the PoC	Success metrics (initial targets)
• Answer questions from the document corpus with verifiable citations. • Decline explicitly when evidence is weak instead of guessing. • Make every configuration change measurable before release. • Know the cost of every answer.	• Multi-tenant isolation and per-tenant corpora. • Conversation memory across turns. • Actions on other systems (the assistant answers, it does not act). • Fine-tuning of any model.	• Faithfulness, judge score: at least 0.90 on the holdout set. • Retrieval recall at 5: at least 0.85. • Grounded-answer rate: tracked; threshold tuned, not maximised. • Latency p95: at most 3 s end to end. • Cost per query: budget set from the baseline run. • Judge agreement with human labels: at least 0.8 before judge scores gate releases. In the PoC the author labels the holdout and agreement is computed against those labels.

Targets are working assumptions until the first baseline evaluation; they are confirmed or revised with data, not by discussion. With a holdout of about twenty questions every rate is reported with its count (for example 18 of 20), not as a percentage alone.

What this proof of concept builds, and what it only designs

The repository is a thin vertical slice that runs locally with Docker Compose. The deployment and operations sections describe where it goes next, so the design is complete without pretending the PoC already runs there.

Capability	Built in the PoC repository	Designed here, built in a pilot
Corpus	A public, licence-clear document set with a financial flavour (for example, public EU payment regulations and consumer finance guidance), a few hundred pages.	Internal policies, FAQs and manuals from the document repository.
Ingestion and indexes	Python script: parse, section-aware chunking with a baseline of about 500 tokens and 10% overlap, in-process embeddings, PostgreSQL with pgvector and full-text search, idempotent by content hash.	Scheduled ECS task reading from object storage.
Retrieval and answer	Hybrid search taking the top 20 from each index, reciprocal rank fusion, cross-encoder reranking to the top 5, confidence floor, generation with citations, citation-coverage check, pinned prompt version.	Same code, unchanged.
Privacy	Regex and pattern based PII redaction before the provider call; embeddings never leave the process.	Named-entity based redaction, EU endpoint under reviewed provider terms, legal sign-off.
Evaluation	A hand-built set of 40 to 60 questions with gold answers and gold chunks, including unanswerable questions and a few prompt-injection cases, development and holdout split, recall@k, MRR, LLM-as-judge, cost and latency per query, thresholds in a config file, one command to run.	Larger labelled set, judge calibration against a human sample, gate wired into CI against staging.
Observability and cost	Structured trace per request written to logs (prompt version, tokens, latency, cost, grounding score); Langfuse export optional.	Langfuse and Grafana dashboards, budget alerts.
API and client	FastAPI with /ask and /health, bearer token, contract tests. No user interface; the four response states are exercised through the API.	Web client with citations, feedback control feeding the evaluation set.
Runtime and delivery	Docker Compose (API, PostgreSQL), unit and integration tests, GitHub Actions running tests and the evaluation command on the development split, to bound judge cost per run.	ECS on Fargate, ALB, RDS, Secrets Manager, Terraform, staging and production.

Figure 1 System context

Who uses the system and what it depends on. The proof of concept owns retrieval, generation, the grounding check and evaluation. Language models and observability are external, replaceable dependencies.

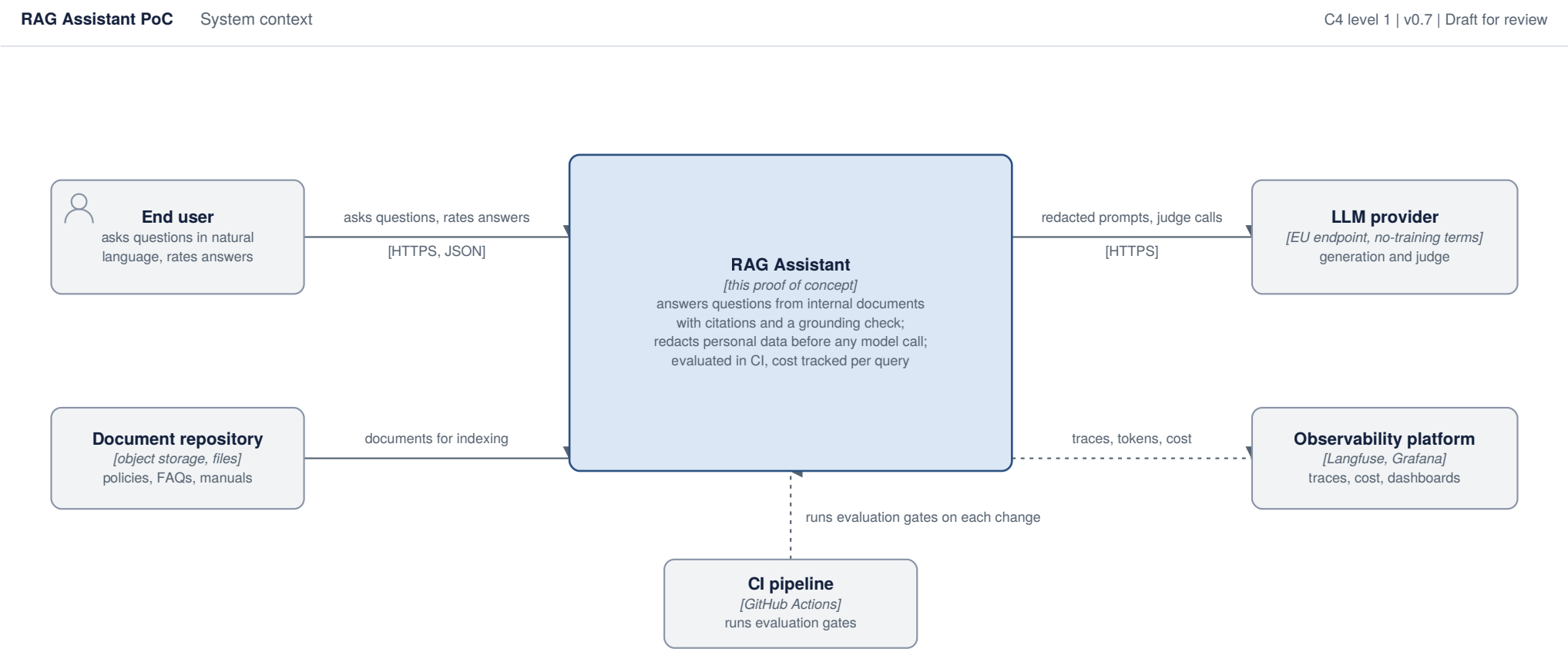


Figure 2 Container view

Deployable units and how they talk. Indexing runs as an idempotent batch job so its cost is paid once. The API service is the only online path, the only place personal data is redacted and the only place the language model is called on the request path. The evaluation runner exercises the same service users hit and calls the judge model separately.

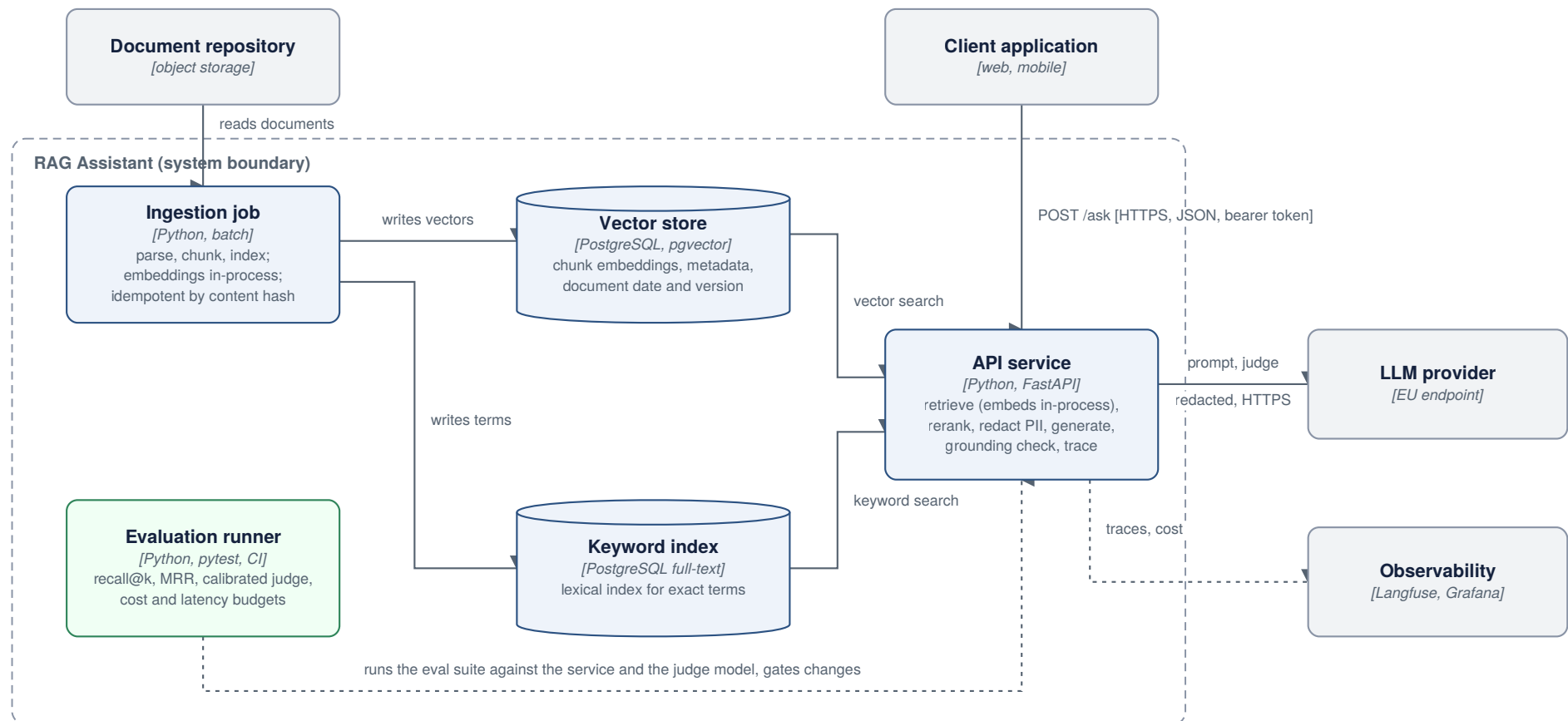


Figure 3 Deployment view

Target deployment for a pilot; the PoC itself runs locally with Docker Compose. Everything stateful sits in private subnets with no public address; the load balancer is the only public entry point and the provider endpoint is the only allowed egress. Environments are separate stacks from one infrastructure-as-code module, and the pipeline deploys immutable image tags.

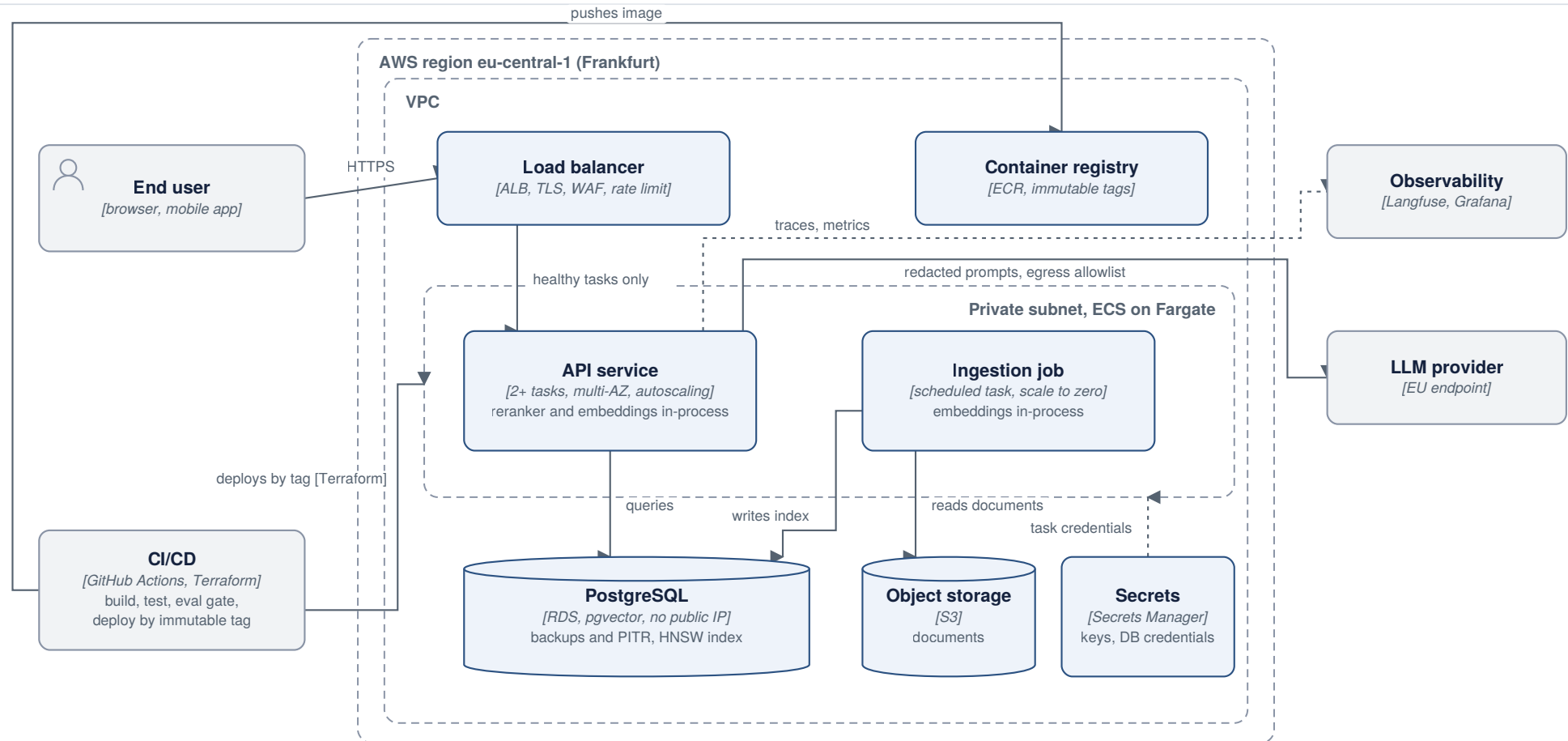
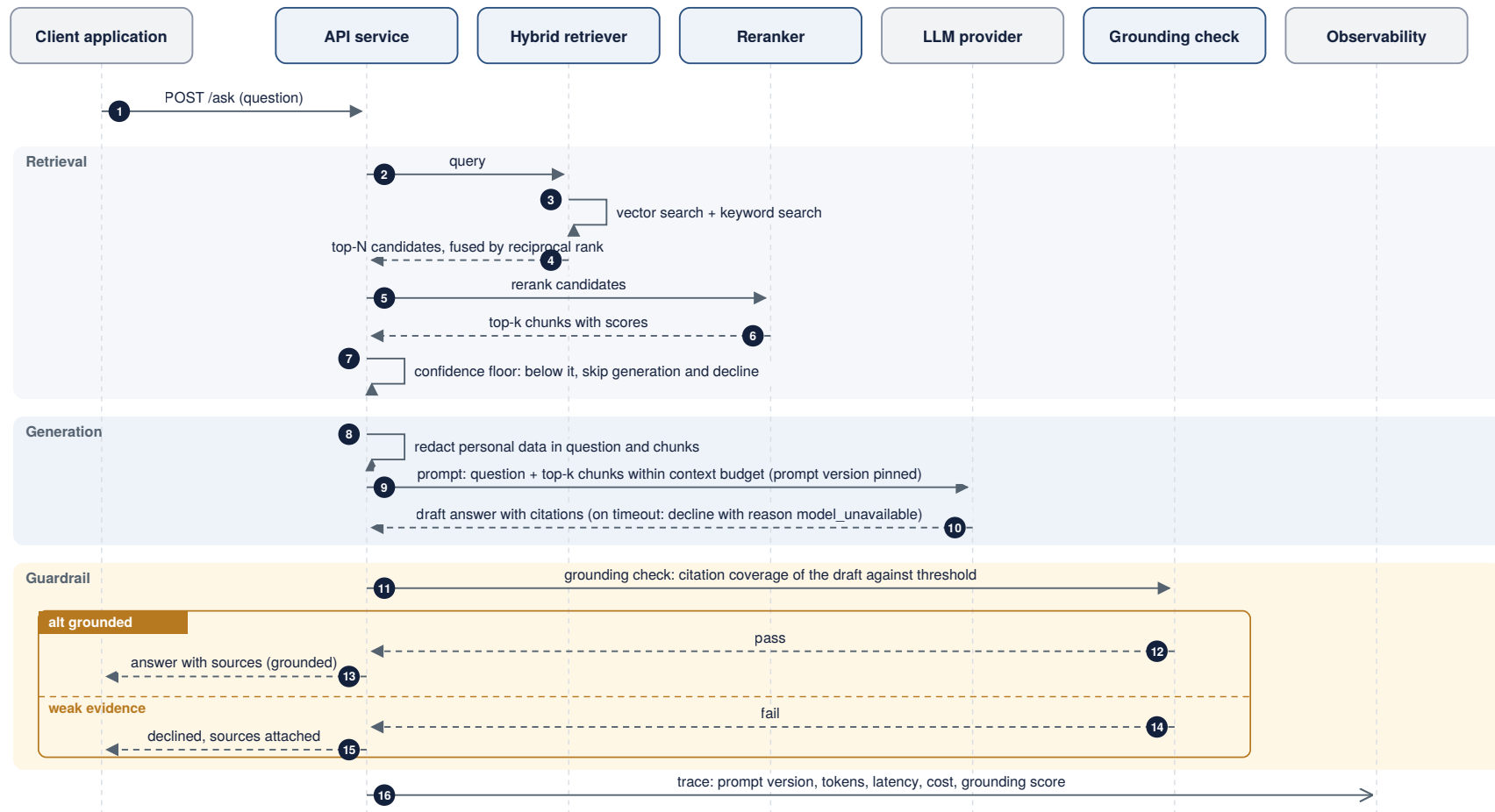


Figure 4 Runtime view: one request

Retrieval, generation and the guardrail are separate phases so each can be measured and tuned on its own. A retrieval confidence floor skips generation entirely when the best results are weak, so no model cost is spent on a question the corpus cannot answer. Personal data is redacted before anything reaches the model. A provider timeout returns the same declined shape with an explicit reason. Every request emits a trace.

RAG Assistant PoC Runtime view: one request

UML sequence | v0.7 | Draft for review



2 User-facing response states

Four states, each with a clear next action. The feedback control on every answer feeds the evaluation dataset, so the people using the assistant improve the gate that protects them.

State	What the user sees	Next action offered
Grounded answer	The answer with inline citation markers, a source list with document title, section and date, and a confidence note derived from the grounding score.	Open a source; rate the answer (helpful or not, with an optional reason).
Insufficient evidence	A plain statement that the documents do not support a confident answer, followed by the closest sources found. No partial or hedged answer is shown.	Rephrase the question; open a source; escalate to a person with the sources attached.
Model unavailable	The answer is withheld and the reason is stated plainly; the closest sources are still shown, so the user can read them directly.	Retry; open a source; escalate with the sources attached.
Service error	A short, specific message with a reference id. No sources, because the service could not retrieve them.	Retry; contact support quoting the reference id.

Streaming of the answer text is a planned improvement to reduce perceived latency; it does not change the states above.

3 API contract

One endpoint. A declined answer is a successful response with `grounded: false` and an explicit `reason`, not an error: the client always receives the sources and the trace id, whether the corpus was insufficient or the model did not respond.

```

POST /ask                                Authorization: Bearer <token>
{ "question": "string", "top_k": 5 }      top_k optional, default 5

200 OK
{
  "answer": "string | null",              null when grounded is false
  "grounded": true,
  "reason": null,                        "insufficient_evidence" | "model_unavailable" when grounded is false
  "retrieval_confidence": 0.71,          reranker score of the best chunk, drives the confidence floor
  "grounding_score": 0.82,               citation coverage of the answer, null when generation was skipped
  "sources": [
    { "id": "policy-142#s3", "title": "string", "section": "string", "version": "3",
      "date": "2026-03-01", "snippet": "string", "score": 0.82 }
  ],
  "prompt_version": "1.3.0",
  "trace_id": "uuid"
}

400  invalid request          401  missing or invalid token
429  rate limit exceeded      503  service unavailable, database or dependency down (trace_id included)

```

Annotated for reading: the comments to the right are not part of the payload.

Figure 5 Governance and quality gates

How changes reach production. Prompts and retrieval settings are versioned like code, evaluated in CI against a fixed dataset with separate development and holdout splits, and released only when retrieval, answer quality and cost budgets pass. Production failures and user feedback flow back into the dataset, so the gate gets stricter over time.

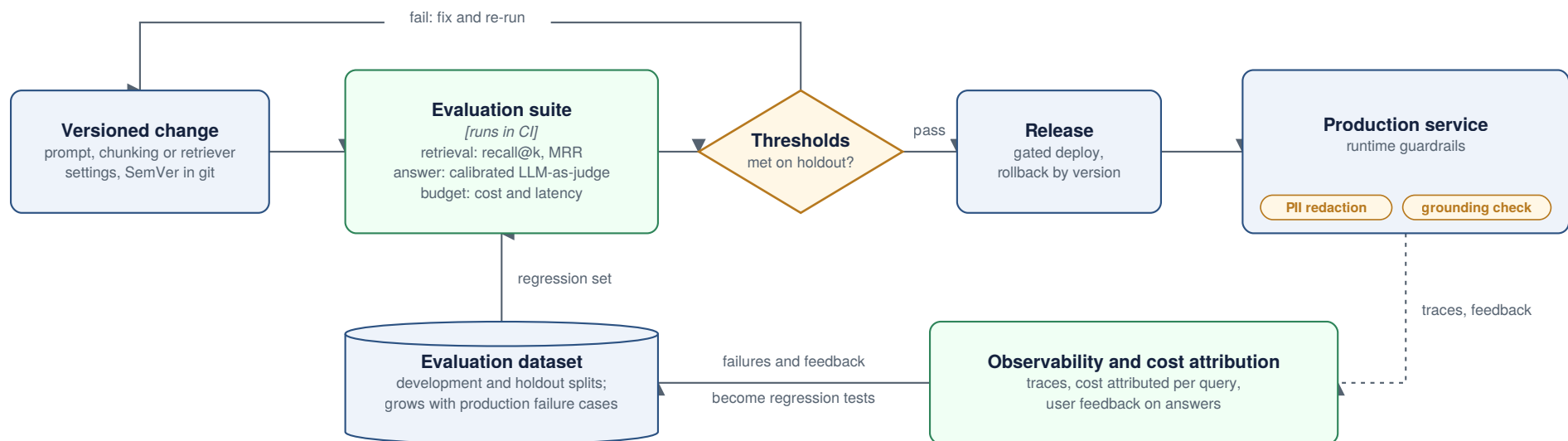


Table 1 **Design decisions**

Each decision lists the options considered and the trade-off accepted. These are the choices the evaluation loop is designed to revisit with data.

Decision	Options considered	Choice and rationale	Trade-off accepted
Retrieval strategy	Vector only; keyword only; hybrid	Hybrid with reciprocal rank fusion. Semantic search covers paraphrase; keyword search covers exact terms such as product names, codes and amounts, which matter in a financial context.	Two indexes to keep consistent and a fusion step to tune.
Reranking	None; cross-encoder; LLM reranker	Cross-encoder over the fused top-N. Higher precision in the final context at a fraction of the cost and latency of an LLM reranker.	Added latency per request and one more model to version.
Answering policy	Always answer; grounding threshold; human in the loop	Two stages. A retrieval confidence floor skips generation when the top results are weak, so no model call is spent. After generation, a citation-coverage check verifies that every claim maps to a retrieved chunk. Either failure returns an explicit declined response with the best sources.	Some answerable questions are declined. Both thresholds are tuned on the holdout set.
Privacy boundary	Send raw text; redact before the model; self-hosted model	Redact personal data from the question and the retrieved chunks before any call to the provider, using an EU endpoint under no-training terms. Keeps a hosted model viable for the PoC.	Redaction can remove useful context; residual risk needs legal review before a pilot.
Embedding model	Hosted embedding API; open-weights model in-process	Open-weights multilingual embedding model running inside the ingestion job and the API. Raw document text never leaves the VPC; the only text that reaches the provider is the redacted prompt.	Quality ceiling below the best hosted embeddings and CPU cost at indexing time; revisited with the evaluation suite.
Vector store	pgvector; dedicated vector database; managed service	PostgreSQL with pgvector. One database for embeddings, metadata and full-text search, transactional updates, and operational simplicity.	Scale ceiling below a dedicated vector database; revisit if corpus or traffic grows.
Evaluation integrity	Single eval set; dev and holdout split; human review only	Development and holdout splits; tuning happens on development, release decisions on holdout. The judge is a different model from the generator and is calibrated against a human-labelled sample before its scores are trusted; in the PoC that sample is the author-labelled holdout.	Judge calls have a cost; a labelled sample must be maintained.

Decision	Options considered	Choice and rationale	Trade-off accepted
Model and prompt versioning	Floating latest; pinned versions	Generation model, judge model, embedding model and prompts are pinned and recorded in every trace. A change to the embedding model triggers a full reindex.	Upgrades are deliberate work, never automatic.
Cost control	No attribution; per-query attribution	Tokens and cost attributed to every query and aggregated per document and section; per tenant once multi-tenant. Budgets are part of the release gate, not an afterthought.	Instrumentation overhead on every request.

Table 2 **Non-functional requirements**

What the service must guarantee beyond answering correctly, and how each is verified. Targets describe the pilot deployment; the PoC demonstrates each mechanism locally.

Requirement	Target	How it is met and verified
Latency	p95 at most 3 s end to end	Measured from traces per phase (retrieval, rerank, generation, guardrail); the latency budget is part of the CI gate.
Availability	Best effort in the PoC; 99.5% target for a pilot	Stateless API behind a health check; indexes in PostgreSQL; provider timeouts and retries with a bounded budget, then a declined answer with reason <code>model_unavailable</code> and the sources still returned.
Security and privacy	No personal data reaches the model; every query attributable	Bearer token authentication; embeddings computed in-process so document text stays inside the VPC; PII redaction before provider calls; secrets from environment variables in the PoC and a secrets manager in the pilot, never in the repository; queries logged with trace id for audit.
Data residency	Data and processing in the EU	Database and API hosted in the EU; provider EU endpoint under no-training terms; only redacted text leaves the boundary. Open question for legal review before a pilot.
Cost	Budget per query set from baseline	Tokens and cost attributed per query; budget breaches fail the CI gate and alert in production.
Reproducibility	Same input, same retrieval and same evaluation	Retrieval and ranking are deterministic. Generation and judging run at temperature 0 with pinned models, which minimises but does not eliminate provider-side variance; eval reports therefore record model versions and run date, and thresholds leave margin for it.
Testing	Every change gated	Unit tests for chunking, fusion and redaction; integration tests against PostgreSQL in Docker; contract tests for /ask; evaluation gates in CI.

Table 3 **Infrastructure and operations**

How the pilot deployment is provisioned, deployed, secured and kept running; the PoC itself runs with Docker Compose. AWS is the reference platform; every element has a direct equivalent on other clouds.

Area	Approach for the pilot	Why it matters
Environments and IaC	Development, staging and production as separate stacks from one Terraform module; every change through a reviewed plan in a pull request; no manual console changes.	Environments stay identical and reproducible; drift is visible in review.
Compute and scaling	API on ECS Fargate, minimum two tasks across availability zones, autoscaling on CPU and p95 latency. Ingestion as a scheduled task that scales to zero. Reranker and embedding model run in-process on CPU.	Online path is redundant; batch work costs nothing while idle.
Data	Managed PostgreSQL with pgvector in a private subnet, automated backups with point-in-time recovery copied to a second region, HNSW index for vectors, connection pooling. A reindex is a job, not a migration.	One stateful system to protect, and it is protected by default.
Network and access	Load balancer terminates TLS with WAF and rate limiting; tasks in private subnets; egress allowlist to the provider EU endpoint only; secrets from a secrets manager. Least privilege per task role: the API reads indexes and calls the provider; the ingestion job writes indexes and never reaches the provider, since embeddings are computed in-process.	Limits blast radius; a compromised task cannot reach what it does not need.
CI/CD pipeline	Build, unit and integration tests, evaluation gate against staging, then deploy an immutable image tag. Rollback is redeploying the previous tag; prompt and model versions travel with the image.	Every production version is reproducible and reversible in minutes.
Monitoring and alerting	Health and readiness endpoints; alerts on error rate, p95 latency, cost budget and database storage; Grafana dashboards per phase; log retention defined per environment.	Problems are noticed by the team, not by users.
Infrastructure cost	Right-sized for a pilot: tasks sized for the in-process embedding and reranker models, a small database instance, scale-to-zero jobs, and a monthly estimate reported next to token cost.	Total cost of an answer includes the platform, not only the model.

Table 4 **Risks and open questions**

Risk	Impact	Mitigation or decision needed
Judge disagrees with human reviewers	Gate passes bad answers or blocks good ones	Calibrate the judge on a labelled sample and report agreement before trusting its scores; keep the human sample current.
Users lose trust after repeated declines	Adoption drops	Track the grounded-answer rate by document and section; tune the threshold on holdout; make the decline state useful (sources and escalation).
Provider model updates change behaviour	Silent quality regression	Pin versions; run the evaluation suite as a canary before adopting any update; reindex plan for embedding changes.
Redacted text still leaves the EU boundary	Compliance exposure	Decision needed: legal review of provider terms and redaction coverage before any pilot with real documents.
Single region, no disaster recovery in the PoC	Extended outage on a regional failure	Accepted for the PoC; backups are cross-region. Decision needed: RTO and RPO targets before a pilot; multi-AZ is already in place and cross-region can follow.
Prompt injection through indexed documents	A crafted document steers answers or leaks context	Retrieved text is treated as data: chunks are delimited and the prompt instructs the model to ignore instructions found in them; active content is stripped at ingestion; the evaluation set includes injection cases.
Stale or conflicting documents in the corpus	Confident but outdated answers	Store document date and version in metadata; show dates in citations; prefer newer versions at fusion time.